Basics

Run and modules

Run

`node app.js`	run a file
`node`	open REPL
`node --watch app.js`	restart on change
`node -v`	check version
`console.log("Hi")`	print
`process.argv`	cli arguments
`process.env.KEY`	env variable
`process.exit(0)`	quit

Modules

`const fs = require("fs")`	CommonJS import
`module.exports = fn`	CommonJS export
`import fs from "fs"`	ES module import
`export default fn`	ES module export
`import { join } from "path"`	named import
`"type": "module"`	enable ESM

Globals

`__dirname  __filename`	CommonJS paths
`import.meta.url`	ESM file URL
`setTimeout  setInterval`	timers

Modules: CJS & ESM

Import and export

CommonJS

`const fs = require("fs");`	import a module
`module.exports = fn;`	export one thing
`module.exports = { a, b };`	export several
`exports.add = add;`	add one export
`require("./file")`	load local file
`require.resolve("x")`	find module path

ES Modules

`import fs from "fs";`	default import
`import { join } from "path";`	named import
`import * as os from "os";`	whole namespace
`export const a = 1;`	named export
`export default fn;`	default export
`await import("./x.js")`	dynamic import

Setup & Paths

`"type": "module"`	use ESM in .js files
`.mjs  .cjs`	force ESM / CJS
`import.meta.url`	current file URL
`fileURLToPath(import.meta.url)`	URL to path
`with { type: "json" }`	import JSON file

File System

fs and path

fs (promises)

`import fs from "fs/promises"`	async fs
`await fs.readFile(p, "utf8")`	read file
`await fs.writeFile(p, data)`	write file
`await fs.appendFile(p, data)`	append to file
`await fs.readdir(dir)`	list folder
`await fs.mkdir(dir, { recursive: true })`	make folders
`await fs.rm(p, { recursive: true })`	delete
`await fs.stat(p)`	file info
`fs.existsSync(p)`	path exists?

path

`path.join(a, b)`	build a path
`path.resolve("a")`	absolute path
`path.basename(p)  path.dirname(p)`	name / folder
`path.extname(p)`	file extension

os

`os.platform()  os.cpus()`	system info
`os.homedir()  os.tmpdir()`	common folders

Process & Env

Talk to the system

Process Info

`process.argv`	command-line arguments
`process.argv.slice(2)`	only your arguments
`process.cwd()`	working folder
`process.platform`	win32, linux, darwin
`process.version`	node version
`process.memoryUsage()`	memory stats

Environment

`process.env.NAME`	read variable
`process.env.PORT ?? 3000`	with default
`node --env-file=.env app.js`	load .env file
`process.loadEnvFile()`	load .env in code

Exit & Signals

`process.exit(1)`	quit with code
`process.exitCode = 1`	set code, exit later
`process.on("SIGINT", fn)`	handle Ctrl+C
`process.on("exit", fn)`	run on exit
`process.stdout.write("hi")`	print without newline
`process.nextTick(fn)`	run before next event

Readline & CLI

Command-line tools

Read Input

`import rl from "node:readline/promises"`	promise readline
`readline.createInterface({ input, output })`	open a prompt
`const a = await rl.question("Name? ");`	ask a question
`rl.close();`	stop reading
`for await (const line of rl) { }`	read line by line

Arguments

`const [,, a, b] = process.argv;`	read arguments
`import { parseArgs } from "node:util";`	parse flags
`parseArgs({ options })`	define flags

Executable Scripts

`#!/usr/bin/env node`	shebang line
`"bin": { "tool": "./cli.js" }`	command in package.json
`npm link`	test your CLI locally
`npx tool`	run without installing
`console.error("msg")`	print to stderr
`process.stdin.isTTY`	interactive terminal?

HTTP & Express

Web servers

http

`http.createServer((req, res) => { })`	basic server
`res.writeHead(200, {...})`	status + headers
`res.end("Hi")`	send response
`server.listen(3000)`	start server

Express

`npm i express`	install
`const app = express()`	create app
`app.use(express.json() )`	parse JSON body
`app.get("/", (req, res) => { })`	GET route
`app.post("/add", handler)`	POST route
`req.params  req.query  req.body`	request data
`res.json({ ok: true })`	send JSON
`res.status(404).send("Nope")`	set status
`app.use(express.static("public"))`	serve files
`app.listen(3000)`	start app

Express Routing

Routes and middleware

Routes

`app.post("/a", handler)`	POST route
`app.put("/a/:id", handler)`	PUT route
`app.delete("/a/:id", handler)`	DELETE route
`app.all("/a", handler)`	any method
`app.route("/a").get(f).post(g)`	chain methods
`const r = express.Router();`	group routes
`app.use("/api", r);`	mount a router

Middleware

`app.use(express.json())`	parse JSON body
`express.urlencoded({ extended: true })`	parse form body
`app.use((req, res, next) => { next(); })`	custom middleware
`app.use(cors())`	allow other origins
`app.use(morgan("dev"))`	request logging

Response

`res.json({ ok: true })`	send JSON
`res.redirect("/home")`	redirect
`res.sendFile(path)`	send a file
`res.set("X-A", "1")`	set a header
`res.cookie("a", "1")`	set a cookie

npm & Packages

Dependencies

npm

`npm init -y`	create package.json
`npm install pkg`	add dependency
`npm i -D pkg`	dev dependency
`npm i -g pkg`	global install
`npm uninstall pkg`	remove package
`npm update`	update packages
`npm run dev`	run script
`npm ci`	clean install

package.json

`"scripts": { "start": "node app.js" }`	custom commands
`"dependencies"`	needed to run
`"devDependencies"`	needed to build
`"main": "index.js"`	entry file
`npx pkg`	run without installing

Security & Deploy

Ship safely

Security Basics

`npm audit`	check dependencies
`npm audit fix`	apply safe fixes
`helmet()`	secure Express headers
`express-rate-limit`	stop abuse
`bcrypt.hash(pw, 10)`	hash passwords
`Validate all input`	never trust the client

Secrets & Config

`.env`	local secrets file
`.gitignore: .env`	never commit secrets
`process.env.KEY`	read in code
`NODE_ENV=production`	production mode

Deploy

`npm ci`	clean install for servers
`npm start`	run production script
`pm2 start app.js`	keep alive and restart
`Dockerfile: FROM node:lts`	container image
`GET /health`	health check route
`process.on("SIGTERM", fn)`	shut down cleanly

Async & Events

Callbacks, promises, streams

Async

`fs.readFile(p, (err, data) => { })`	callback style
`await promise`	wait for result
`util.promisify(fn)`	callback to promise
`Promise.all([a, b])`	run in parallel
`process.nextTick(fn)`	run very soon

Events

`import { EventEmitter } from "events"`	events module
`const e = new EventEmitter()`	create emitter
`e.on("hi", fn)`	listen
`e.emit("hi", data)`	trigger
`e.once("hi", fn)`	listen once

Streams

`fs.createReadStream(p)`	read in chunks
`stream.pipe(res)`	pipe data
`readline.createInterface(...)`	read lines

Timers & Workers

Run later and in parallel

Timers

`setTimeout(fn, 1000)`	run once later
`setInterval(fn, 1000)`	repeat
`setImmediate(fn)`	after I/O events
`clearTimeout(id)`	cancel timer
`await setTimeout(1000)`	promise sleep (timers/promises)
`timer.unref()`	do not keep process alive

Event Loop Order

`process.nextTick`	runs first
`Promise.then`	microtask next
`setTimeout`	timer phase
`setImmediate`	check phase

Threads & Processes

`new Worker("./w.js")`	run in a thread
`parentPort.postMessage(x)`	send from worker
`worker.on("message", fn)`	receive in main
`spawn("ls", ["-la"])`	run a command
`execSync("git status")`	run and wait
`cluster.fork()`	one process per core

Errors & Debugging

Find and fix

Errors

`try { await f(); } catch (e) { }`	handle async errors
`throw new Error("msg");`	throw an error
`err.message  err.stack`	message / trace
`new Error("x", { cause: e })`	chain the cause
`process.on("uncaughtException", fn)`	last-resort handler
`process.on("unhandledRejection", fn)`	missed promise error

Express Errors

`app.use((err, req, res, next) => { })`	error middleware
`res.status(500).json({ error })`	send error reply
`next(err)`	pass error on

Debugging

`node --inspect app.js`	open debugger
`node --inspect-brk app.js`	pause at first line
`debugger;`	breakpoint in code
`console.trace()`	print call stack
`console.table(data)`	table output
`NODE_DEBUG=http node app.js`	built-in debug logs

Testing

Prove it works

Built-in Runner

`import test from "node:test";`	test module
`test("adds", () => { });`	define a test
`describe("group", () => { });`	group tests
`node --test`	run all tests
`node --test --watch`	rerun on change
`node --test --experimental-test-coverage`	coverage report

Assertions

`import assert from "node:assert";`	assert module
`assert.equal(a, b);`	loose equal
`assert.strictEqual(a, b);`	strict equal
`assert.deepStrictEqual(a, b);`	compare objects
`assert.throws(() => f());`	must throw
`await assert.rejects(p);`	promise must fail

Mock & Tools

`mock.fn()`	fake function
`mock.method(obj, "m")`	fake a method
`npm i -D jest  vitest`	popular runners
`npm i -D supertest`	test HTTP routes

Databases

Store data

SQLite (built-in)

`import { DatabaseSync } from "node:sqlite"`	built-in SQLite
`const db = new DatabaseSync("a.db");`	open a database
`db.exec("CREATE TABLE t (id INT)");`	run SQL
`db.prepare("INSERT ...").run(1)`	insert safely
`db.prepare("SELECT * FROM t").all()`	read rows

MongoDB

`npm i mongodb`	install driver
`new MongoClient(url)`	connect
`db.collection("a").insertOne(doc)`	add one
`db.collection("a").find({}).toArray()`	read all
`col.updateOne(f, { $set: u })`	update

SQL & ORMs

`npm i pg`	PostgreSQL client
`npm i mysql2`	MySQL client
`npm i prisma`	popular ORM
`Use ? placeholders`	never join SQL strings
`Close connections`	avoid leaks

Tips & Tools

Handy extras

Built-ins

`crypto.randomUUID()`	unique id
`fetch(url)`	built-in fetch
`Buffer.from("hi")`	binary data
`child_process.exec("ls")`	run command
`worker_threads`	run in parallel
`node:test`	built-in test runner

Popular Packages

`express`	web server
`dotenv`	load .env file
`nodemon`	auto restart
`axios`	HTTP client
`jest / vitest`	testing

Good Habits

`Never commit .env or secrets`	stay safe
`Handle errors in async code`	no crashes
`Use LTS versions`	stable runtime

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily